

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : CSA0923- Programming in JAVA

Assignment Title:	A Java-based Warehouse Management System using object-oriented programming concepts.
Course Outcome(s) – CO:	CO1 Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations CO2: Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications. CO3: Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 8 – Decent Work and Economic Growth SDG 9 – Industry, Innovation and Infrastructure

Submitted by

Akshaya A(192421222)

Monisha G(192411020)

Ch Madhu Lahhari(192472126)

Problem Statement

Design and implement an object-oriented Java architecture for an Autonomous Warehouse Management System. Identify the major entities and establish appropriate classes and objects to represent robots, warehouse items, storage locations, and tasks. Apply encapsulation and data hiding to protect sensitive attributes such as battery level, robot status, and item information. Use inheritance to derive specialized robot classes from a common Robot class and apply polymorphism to enable different robot types to perform common warehouse tasks in different ways. Demonstrate the interaction between the objects and analyze how the proposed architecture can be extended to support additional robot types and warehouse operations without significantly modifying the existing system.

Objective

The main objective of this project is to design and implement an Autonomous Warehouse Management System using Java and Object-Oriented Programming concepts. The system aims to represent warehouse robots, items, storage locations and tasks as objects and demonstrate how these objects interact.

The specific objectives are:

- To implement classes and objects for warehouse entities.
- To apply encapsulation and data hiding for sensitive information such as robot battery level and status.
- To use inheritance to create specialized robot classes.
- To demonstrate polymorphism and method overriding for different robot behaviours.

- To use collections to manage multiple robot objects.
- To handle invalid operations using exception handling.

Requirements and Constraints

Functional Requirements

- Create and manage warehouse robots.
- Store warehouse item information and quantities.
- Represent storage locations.
- Create and assign tasks to robots.
- Demonstrate different robot behaviours using polymorphism.
- Update robot battery and status through controlled methods.
- Display the final warehouse operation results.

Technical Constraints

- Use Java for implementation.
- Apply classes, objects, encapsulation, inheritance and polymorphism.
- Protect sensitive attributes using private access.
- Use method overriding for specialized robot behaviour.
- Program must compile and execute using the Java compiler.

1. Problem Understanding and Formulation

An autonomous warehouse contains items stored at different locations and robots that perform material-handling tasks. A task connects a robot with an operation such as picking or transporting an item. The challenge is to model these real-world entities as Java classes while maintaining data security, code reuse and extensibility.

Expected outcome: a console-based Java application that creates warehouse objects, assigns tasks to different robot types and demonstrates different behaviours through runtime polymorphism.

Major Entities

- Robot – common parent class containing robot ID, battery level and status.
- PickingRobot – specialized robot for picking warehouse items.
- TransportRobot – specialized robot for transporting items.
- WarehouseItem – represents an item with ID, name and quantity.
- StorageLocation – represents the location where an item is stored.
- WarehouseTask – represents an operation assigned to a robot.

Object Relationship

Robot is the superclass of PickingRobot and TransportRobot. WarehouseTask uses a Robot object and operates on a WarehouseItem. WarehouseItem is associated with a StorageLocation.

2. Application of Course Knowledge

The implementation applies classes and objects, encapsulation, data hiding, inheritance, method overriding, runtime polymorphism, collections and exception handling.

Concept	Application	Example
Class & Object	Represent warehouse entities as Java classes and objects.	Robot r1
Encapsulation	Keep data private and access it through methods.	private double batteryLevel
Data Hiding	Protect sensitive robot information.	getBattery()
Inheritance	Derive specialized robots from Robot.	PickingRobot extends Robot
Polymorphism	Use a parent reference for different robot behaviours.	robot.performTask()
Method Overriding	Child classes implement task behaviour differently.	@Override performTask()
Collection	Store multiple robot objects.	ArrayList<Robot>
Exception Handling	Handle invalid operations safely.	try-catch

3. Solution / Design / Methodology

The system uses a simple object-oriented design. Robot contains common robot data and behaviour. PickingRobot and TransportRobot extend Robot and override performTask(). WarehouseItem and StorageLocation model inventory information. WarehouseTask connects a robot with an item and executes the required operation.

Class Design

- Robot: robotId, batteryLevel, status; methods getBattery(), setStatus(), charge(), performTask().
- PickingRobot: overrides performTask() for picking items.
- TransportRobot: overrides performTask() for transporting items.
- WarehouseItem: itemId, itemName, quantity; methods getQuantity(), removeQuantity().
- StorageLocation: locationId, zone; method getId().
- WarehouseTask: taskId, robot, item; method execute().

Pseudo Code

START

Create warehouse items and storage locations

Create PickingRobot and TransportRobot

Store robots using Robot references

Create warehouse tasks

For each task:

 Check robot battery

 If available, execute performTask()

 Update robot status and battery

Display results

STOP

Algorithm

1. Initialize warehouse items and storage locations.
2. Create different robot objects using subclasses of Robot.
3. Create warehouse tasks for picking and transporting.
4. Store robots using the common Robot type.
5. Execute tasks through Robot references.
6. Use overridden performTask() methods for runtime polymorphism.
7. Update battery and status through encapsulated methods.
8. Display final results.

4. Use of Modern Tools

- Java JDK – compilation and execution.
- VS Code / IntelliJ IDEA / Eclipse – source-code development.
- Command Prompt / Terminal – compiling and running the program.
- Git and GitHub – version control and submission.

5. Results and Validation

The implementation creates warehouse items, storage locations and two specialized robots. Tasks are assigned through the common Robot type. When performTask() is called, the correct overridden method executes according to the actual robot object.

Sample Output

=== AUTONOMOUS WAREHOUSE MANAGEMENT SYSTEM ===

Item: Laptop | Quantity: 20 | Location: A-01

Item: Keyboard | Quantity: 50 | Location: B-02

PickingRobot R101: Picking Laptop from A-01

Task T001 completed.

Battery: 80.0%

TransportRobot R102: Transporting Keyboard to B-02

Task T002 completed.

Battery: 70.0%

All warehouse tasks processed.

Validation shows that encapsulation prevents direct access to sensitive attributes, inheritance provides reusable robot functionality, and runtime polymorphism produces different behaviours for different robot types.

7. Test Cases and Expected/Actual Results

Test Case 1 – Creating Warehouse Items

Input: Create a Laptop with quantity 20 and a Keyboard with quantity 50.

Expected Result: The items should be created successfully with their respective storage locations.

Actual Result: Laptop and Keyboard were created successfully.

Status: Pass.

Test Case 2 – Creating Robots

Input: Create PickingRobot R101 and TransportRobot R102.

Expected Result: Both robots should be created with their specified battery levels.

Actual Result: Both robots were created successfully.

Status: Pass.

Test Case 3 – Picking Task

Input: Assign a picking task to PickingRobot R101 for the Laptop.

Expected Result: R101 should perform the picking operation and its battery level should decrease.

Actual Result: R101 successfully performed the picking operation.

Status: Pass.

Test Case 4 – Transport Task

Input: Assign a transport task to TransportRobot R102 for the Keyboard.

Expected Result: R102 should perform the transport operation and its battery level should decrease.

Actual Result: R102 successfully performed the transport operation.

Status: Pass.

Test Case 5 – Polymorphism

Input: Execute performTask() using a Robot reference.

Expected Result: The appropriate overridden method should execute according to the actual robot type.

Actual Result: PickingRobot and TransportRobot performed their respective operations.

Status: Pass.

Test Case 6 – Empty Battery

Input: Attempt to execute a task when the robot battery level is 0.

Expected Result: The task should not execute and an error should be displayed.

Actual Result: The error was handled using exception handling.

Status: Pass.

Test Case 7 – Insufficient Stock

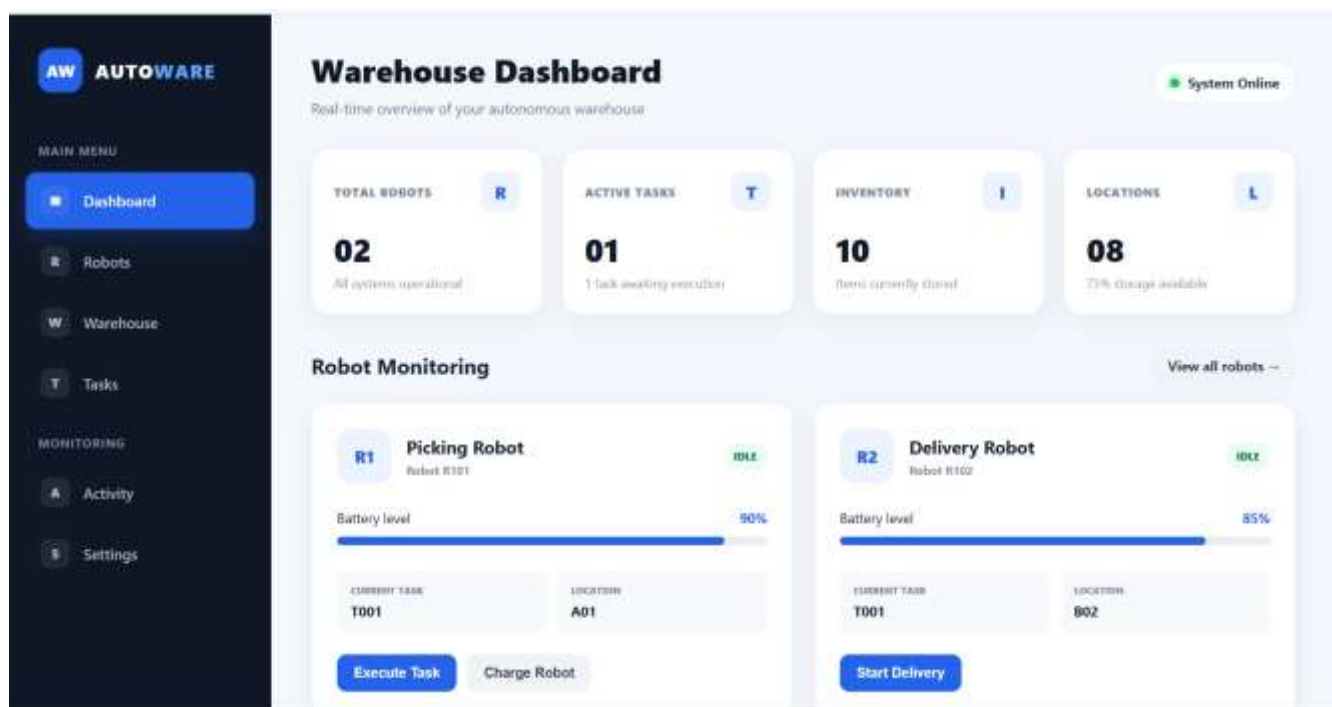
Input: Attempt to remove more items than the available quantity.

Expected Result: The operation should be rejected and an exception should be generated.

Actual Result: The invalid operation was handled using exception handling.

Status: Pass.

Implementation:



6. Analysis and Engineering Decision

Approach	Advantage	Limitation
Single Robot class	Simple	Cannot represent different robot behaviours effectively
Inheritance + overriding	Reusable and supports specialized behaviour	Requires a suitable class hierarchy
Inheritance + polymorphism + collection	Flexible, reusable and extensible	Slightly more design effort

The inheritance, overriding and polymorphism approach is selected because it satisfies the assignment requirements while keeping common functionality in the Robot superclass. A new robot can be added by extending Robot and overriding perform Task(), without changing the existing task-processing logic.

7. Broader Considerations

- Sustainability: automation can improve resource utilization and reduce unnecessary movement.
- Safety: robot status and task assignment should be controlled to avoid conflicting operations.
- Economics: automation can improve productivity in repetitive warehouse activities.
- Professional responsibility: software should protect operational data and handle invalid operations safely.

Conclusion

The Autonomous Warehouse Management System was designed using Java object-oriented programming concepts. Robots, items, storage locations and tasks are represented as interacting objects. Encapsulation protects sensitive information, inheritance provides code reuse and polymorphism allows different robot types to perform common operations differently. The design is modular and can be extended with new robot types and warehouse operations.

Individual Contribution of Group Members:

Each group member contributed to the development of the Autonomous Warehouse Management System. The team collectively worked on understanding the problem, designing the object-oriented architecture, implementing the Java classes, testing the system and preparing the documentation. The members contributed to different activities such as designing the Robot, WarehouseItem, StorageLocation and WarehouseTask classes, implementing inheritance and polymorphism, handling exceptions, testing the program with different scenarios, capturing execution results and preparing the final assignment report and GitHub submission. The combined contribution of all members helped in completing the project successfully.

Student Reflection

This assignment helped in understanding how object-oriented programming concepts can be applied to a practical warehouse automation problem. It provided practical experience with classes, objects, constructors, access control, inheritance, method overriding, polymorphism and collections. With additional time, the system could be extended with a database, graphical interface, real-time robot monitoring and automatic task scheduling.

10. References

- Schildt, H. (2018). *Java: The Complete Reference*, 11th Edition. McGraw-Hill Education.
- Horstmann, C. S. (2019). *Core Java Volume I—Fundamentals*, 11th Edition. Pearson.
- Sierra, K., & Bates, B. (2005). *Head First Java*, 2nd Edition. O'Reilly Media.
- Bloch, J. (2018). *Effective Java*, 3rd Edition. Addison-Wesley Professional.
- Gosling, J., Joy, B., Steele, G., Bracha, G., & Buckley, A. (2023). *The Java Language Specification, Java SE 21 Edition*. Oracle.
- Oracle. (2023). *Java Platform, Standard Edition 21 API Specification*. Oracle Corporation.
- Oracle. (2023). *The Java Tutorials: Object-Oriented Programming Concepts*. Oracle Corporation.

One Page write Up

While developing the Autonomous Warehouse Management System, I learned how object-oriented programming can be used to represent the different activities that take place in a warehouse. Instead of keeping all the operations in one class, we divided the system into different classes such as Robot, Picking Robot, Transport Robot, Warehouse Item, Storage Location, and Warehouse Task. This made the program easier to understand and helped each class have a specific responsibility.

The Robot class was designed as the common parent class because both picking and transport robots share common information such as robot ID, battery level, and status. The Picking Robot and Transport Robot classes inherit these features and provide their own implementation of perform Task(). This was useful because the same method can produce different results depending on the type of robot. We also used encapsulation by keeping important information private and providing methods to access or update it. This prevents unwanted changes to values such as the robot's battery level and status.

Another important design choice was the use of polymorphism. Robots can be handled through a common Robot reference even though they perform different tasks. This reduced the need for separate task-processing logic for every robot type. If the warehouse needs another type of robot in the future, a new class can be created by extending Robot and overriding the required method. Therefore, the existing system does not need to be completely redesigned.

During the implementation, we faced challenges in managing the interaction between robots, items, locations, and tasks. We also had to consider invalid situations, such as trying to perform a task when the robot had no battery or attempting to remove more items than were available. Exception handling was used to manage these situations safely. Testing these cases helped us understand that a good program should not only work under normal conditions but should also respond properly when unexpected inputs occur.

Overall, this project improved my practical understanding of classes, objects, encapsulation, inheritance, polymorphism, collections, and exception handling. More importantly, I understood how these concepts work together to create a modular and extensible application. If the project is developed further, features such as a database, graphical interface, real-time robot monitoring, and automatic task scheduling could be added.